



Polimorfismo

Fundamentos de Lenguajes de Programación – IIC2560

Yadran Eterovic S. (yadran@uc.cl)

Un sistema de tipos en el cual cualquier objeto del lenguaje tiene un único tipo es *monomórfico*

El conjunto de información y reglas que gobierna los tipos (de datos) en un lenguaje:

- el conjunto de tipos predefinidos
- los mecanismos que permiten la definición de nuevos tipos
- los mecanismos para el control de tipos: equivalencia, compatibilidad, inferencia
- chequeo estático o dinámico de restricciones

Un *sistema de tipos* en el cual cualquier objeto del lenguaje tiene un único tipo es monomórfico

Un sistema de tipos en el cual un mismo objeto puede tener más que un tipo es **polimórfico**:

- el lenguaje también es polimórfico

Un objeto es **polimórfico** cuando el sistema de tipos le asigna más que un tipo:

- e.g., en varios lenguajes el nombre `+` es de tipo

$$\text{int} \times \text{int} \rightarrow \text{int}$$

... y también de tipo

$$\text{float} \times \text{float} \rightarrow \text{float}$$

- e.g., la función **length** es de tipo $T[] \rightarrow \text{int}$, para todo tipo T

No siempre el programador puede definir objetos polimórficos:

- e.g., en Pascal, Java, C una función que ordena un vector de enteros tiene la firma

```
void int_sort(int A[])
```

... y una que ordene un vector de caracteres tiene la firma

```
void char_sort(char A[])
```

... y aunque el código de ambas sea idéntico, hay que definirlas como dos funciones diferentes

Un lenguaje polimórfico permitiría la definición de una sola función

```
void sort(<T> A[])
```

... en que <T> denota un tipo genérico que se especifica después

Polimorfismo por overloading

Un nombre está *overloaded* (sobrecargado) cuando corresponde a más de un objeto

... y es necesario usar información estática de contexto para determinar cuál objeto es denotado por una instancia específica del nombre

(Es polimorfismo sólo en apariencia)

Polimorfismo por **overloading**

Un nombre está *overloaded* (sobrecargado) cuando corresponde a más de un objeto

... y es necesario usar información estática de contexto para determinar cuál objeto es denotado por una instancia específica del nombre

(Es polimorfismo sólo en apariencia)

Un pre análisis del programa resuelve los casos de overloading sustituyendo un nombre no ambiguo —que denota de manera única a un objeto único objeto— por cada símbolo overloaded

Ejs. comunes:

- el uso de + para indicar suma entera o real
- definir varias funciones con el mismo nombre, pero cuyas instancias difieren en el número o tipo de sus parámetros

No confundir overloading con *coerción*:

Si hay compatibilidad entre los tipos T y S , el lenguaje permite que aparezca un valor de tipo T cuando se esperaba un valor de tipo S :

- el compilador inserta una conversión de tipos entre T y S

(Desde el punto de vista de implementación, una coerción puede corresponder a cosas diferentes según la noción de compatibilidad adoptada)

No confundir overloading con *coerción*:

Si hay compatibilidad entre los tipos T y S , el lenguaje permite que aparezca un valor de tipo T cuando se esperaba un valor de tipo S :

- el compilador inserta una conversión de tipos entre T y S

(Desde el punto de vista de implementación, una coerción puede corresponder a cosas diferentes según la noción de compatibilidad adoptada)

Un tipo T puede ser compatible con S cuando:

- T y S son equivalentes
- Los valores de T forman un subconjunto de los valores de S ; usualmente, tienen la misma representación interna
- Todas las operaciones sobre los valores de S están permitidas también sobre los valores de T
- Los valores de T corresponden de una manera canónica a los valores de S
- Los valores de T pueden hacerse corresponder con algunos valores de S

No confundir overloading con *coerción*:

Si hay compatibilidad entre los tipos T y S , el lenguaje permite que aparezca un valor de tipo T cuando se esperaba un valor de tipo S :

- el compilador inserta una conversión de tipos entre T y S

(Desde el punto de vista de implementación, una coerción puede corresponder a cosas diferentes según la noción de compatibilidad adoptada)

Un tipo T puede ser compatible con S cuando:

- T y S son equivalentes
- Los valores de T forman un subconjunto de los valores de S ; usualmente, tienen la misma representación interna
- Todas las operaciones sobre los valores de S están permitidas también sobre los valores de T
- Los valores de T corresponden de una manera canónica a los valores de S
- Los valores de T pueden hacerse corresponder con algunos valores de S

No confundir overloading con *coerción*:

Si hay compatibilidad entre los tipos T y S , el lenguaje permite que aparezca un valor de tipo T cuando se esperaba un valor de tipo S :

- el compilador inserta una conversión de tipos entre T y S

(Desde el punto de vista de implementación, una coerción puede corresponder a cosas diferentes según la noción de compatibilidad adoptada)

Si hay coerción, puede no ser claro cómo resolver un caso de overloading

E.g.:

$1 + 2$

$1.0 + 2.0$

$1 + 2.0$

$1.0 + 2$

- $+$ está overloaded con 4 significados distintos
- $+$ está overloaded con dos significados y se inserta coerción en los últimos dos casos
- $+$ denota sólo suma real y en los otros tres casos se usa coerción

Polimorfismo paramétrico (universal)

Un valor que tiene un número infinito de tipos diferentes que pueden ser obtenidos instanciando un único esquema de tipo general

Una función polimórfica universal está compuesta de una única pieza de código que opera uniformemente sobre todas las instancias de su tipo general

Polimorfismo paramétrico (universal)

Un valor que tiene un número infinito de tipos diferentes que pueden ser obtenidos instanciando un único esquema de tipo general

Una función polimórfica universal está compuesta de una única pieza de código que opera uniformemente sobre todas las instancias de su tipo general

La información sobre el tipo no es usada por el algoritmo implementado por la función

Polimorfismo paramétrico (universal)

Un valor que tiene un número infinito de tipos diferentes que pueden ser obtenidos instanciando un único esquema de tipo general

Una función polimórfica universal está compuesta de una única pieza de código que opera uniformemente sobre todas las instancias de su tipo general

La información sobre el tipo no es usada por el algoritmo implementado por la función

E.g., la función

```
void sort(<T> A[])
```

... que ordena un array de cualquier tipo

<T> no es de tipo habitual,
sino una suerte de parámetro

Polimorfismo paramétrico (universal)

Un valor que tiene un número infinito de tipos diferentes que pueden ser obtenidos instanciando un único esquema de tipo general

Una función polimórfica universal está compuesta de una única pieza de código que opera uniformemente sobre todas las instancias de su tipo general

La información sobre el tipo no es usada por el algoritmo implementado por la función

E.g., la función

```
void sort(<T> A[])
```

... que ordena un array de cualquier tipo

<T> no es de tipo habitual, sino una suerte de parámetro

E.g., una función polimórfica que intercambia dos variables de cualquier tipo

```
void swap(ref <T> x, ref <T> y) {  
    <T> tmp = x;  
    x = y;  
    y = tmp;  
}
```

En un lenguaje sin polimorfismo habría que escribir una función **swap** para cada tipo posible de variable

Un objeto polimórfico puede ser *instanciado* a un tipo específico

La instanciación puede ocurrir de varias maneras diferentes, según los mecanismos provistos por el lenguaje

El modelo más simple es uno en el cual ocurre automáticamente, y es realizada por el compilador

Este polimorfismo es *explícito*:

- en el programa hay anotaciones especiales, e.g., `<T>`, que indican los tipos que deben ser considerados parámetros
- e.g., en C++ (“template”) y en Java (“generic”)



```
char v,w;  
int i,j;  
...  
swap(v,w);  
swap(i,j);
```

E.g., una función polimórfica que intercambia dos variables de cualquier tipo

```
void swap(ref <T> x, ref <T> y) {  
    <T> tmp = x;  
    x = y;  
    y = tmp;  
}
```


En lenguajes funcionales, hay polimorfismo *implícito*:

- el programador no provee ninguna indicación especial
- el verificador de tipos del lenguaje, o el módulo que realiza la inferencia de tipos, trata de determinar para cada objeto el tipo más general

En lenguajes funcionales, hay polimorfismo *implícito*:

- el programador no provee ninguna indicación especial
- el verificador de tipos del lenguaje, o el módulo que realiza la inferencia de tipos, trata de determinar para cada objeto el tipo más general

Definición de función

```
fun Ide(x) {return x;}
```

E.g., la función identidad:

La inferencia de tipos le asigna a la función **Ide** el tipo $\langle T \rangle \rightarrow \langle T \rangle$

La aplicación de una función polimórfica ocurre sin indicación de tipos:

- si el argumento es de tipo **X**, entonces el resultado será de tipo **X** para cualquier tipo **X**
- **Ide(3)** será tipificada como **int**;
Ide(true), como **bool**

En lenguajes funcionales, hay polimorfismo *implícito*:

- el programador no provee ninguna indicación especial
- el verificador de tipos del lenguaje, o el módulo que realiza la inferencia de tipos, trata de determinar para cada objeto el tipo más general

```
fun Comp(f,g,x) {return f(g(x));}
```

Comp aplica la composición de sus dos primeros parámetros al argumento **x**

El tipo de datos inferido más general para **Comp** es
 $(\langle S \rangle \rightarrow \langle T \rangle) \times (\langle R \rangle \rightarrow \langle S \rangle) \times \langle R \rangle \rightarrow \langle T \rangle$

Definición de función

```
fun Ide(x) {return x;}
```

E.g., la función identidad:

La inferencia de tipos le asigna a la función **Ide** el tipo $\langle T \rangle \rightarrow \langle T \rangle$

La aplicación de una función polimórfica ocurre sin indicación de tipos:

- si el argumento es de tipo **X**, entonces el resultado será de tipo **X** para cualquier tipo **X**
- **Ide(3)** será tipificada como **int**;
Ide(true), como **bool**

Polimorfismo de subtipo (o acotado)

A un objeto se le puede asignar un número infinito de tipos diferentes, obtenidos instanciando un único tipo más general:

- también hay un único algoritmo que es uniforme en el tipo que puede ser instanciado para cada uno de los tipos posibles

Pero no todas las instanciaciones posibles del esquema para el tipo más general son admisibles:

- están limitadas por la noción de *compatibilidad estructural* entre tipos —i.e., usando el concepto de subtipo

Típicamente, en lenguajes O-O

Polimorfismo de subtipo (o acotado)

A un objeto se le puede asignar un número infinito de tipos diferentes, obtenidos instanciando un único tipo más general:

- también hay un único algoritmo que es uniforme en el tipo que puede ser instanciado para cada uno de los tipos posibles

Pero no todas las instanciaciones posibles del esquema para el tipo más general son admisibles:

- están limitadas por la noción de *compatibilidad estructural* entre tipos —i.e., usando el concepto de subtipo

... i.e., no depende de la estructura particular del tipo

Si hay compatibilidad estructural entre tipos, entonces hay polimorfismo (aunque no tan completamente general como polimorfismo paramétrico)

E.g., una función polimórfica de tipo
para todo **T** que sea un subtipo de **D**, el tipo del recorrido es **void**
... puede ser aplicada a todos los valores de cualquier subtipo de **D**

Para la relación de subtipo, cada instancia de una clase **A** tiene, como tipo, todas las superclases de **A**

Consideremos el método

B `foo(A x) { ... }`

foo puede recibir como argumento un valor de cualquier subclase de **A**:

- el código de **foo** no necesita ser adaptado a subclases específicas
- la estructura de las clases asegura que una operación que puede ser ejecutada sobre valores de tipo **A** también puede ser ejecutada sobre valores de sus subclases
- el tipo del método **foo** es para todo **T** que es un subtipo de **A**, el dominio es de tipo **T** y el recorrido de tipo **B**

Para la relación de subtipo, cada instancia de una clase **A** tiene, como tipo, todas las superclases de **A**

Consideremos el método

```
B foo(A x) { ... }
```

foo puede recibir como argumento un valor de cualquier subclase de **A**:

- el código de **foo** no necesita ser adaptado a subclases específicas
- la estructura de las clases asegura que una operación que puede ser ejecutada sobre valores de tipo **A** también puede ser ejecutada sobre valores de sus subclases
- el tipo del método **foo** es para todo **T** que es un subtipo de **A**, el dominio es de tipo **T** y el recorrido de tipo **B**

Consideremos

```
A Ide(A x) {return x;}
```

Su tipo es para todo **T** que es un subtipo de **A**, el dominio es de tipo **T** y el recorrido de tipo **T**

Suponiendo que **C** es subtipo de **A** y que **c** es una instancia de **C**, ¿qué pasa con

```
C cc = Ide(c);    ?
```

Un chequeador de tipos estático la rechaza

Un stack de elementos representado como una lista ligada:

- no queremos fijar desde un comienzo el tipo de los objetos

⇒ **Object**

- **pop** devuelve un **Object**

```
class Elem{
    Object info;
    Elem prox;
}
class Stack{
    private Elem top = null;
    boolean isEmpty(){
        return top==null;
    }
    void push (Object o){
        Elem ne = new Elem();
        ne.info = o;
        ne.prox = top;
        top = ne;
    }
    Object pop() {
        Object tmp = top.info;
        top = top.prox;
        return tmp;
    }
}
```


Un stack de elementos representado como una lista ligada:

- no queremos fijar desde un comienzo el tipo de los objetos
⇒ **Object**

- **pop** devuelve un **Object**

```
class Elem{
    Object info;
    Elem prox;
}
class Stack{
    private Elem top = null;
    boolean isEmpty(){
        return top==null;
    }
    void push (Object o){
        Elem ne = new Elem();
        ne.info = o;
        ne.prox = top;
        top = ne;
    }
    Object pop() {
        Object tmp = top.info;
        top = top.prox;
        return tmp;
    }
}
```

```
C c;
Stack s = new Stack();
s.push(new C());
c = s.pop();
```

Incorrecta: Tratamos de asignar un **Object** a una variable de tipo **C**:

- podemos corregirlo mediante el cast
c = (C) s.pop();